

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Jeff Rehobot Hasian L Gaol - 5024241073

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: STM32 Blackpill (Blink Arduino)

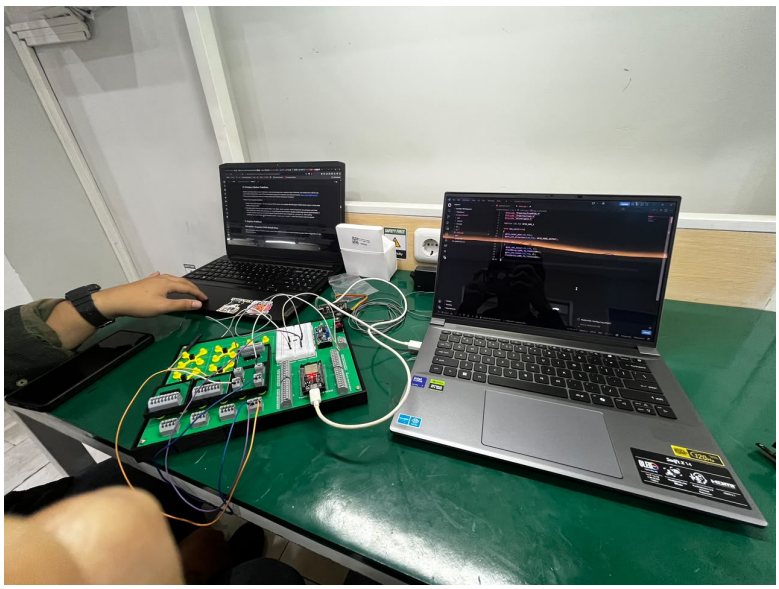
1. Hubungkan ST-Link V2 ke Blackpill (SWDIO, SWCLK, GND, 3V3).
2. Atur `platformio.ini`: board `blackpill_f401cc`, framework `arduino`, upload `stlink`.
3. Tulis kode Blink pada pin PC13 dengan delay 500 ms, lalu unggah (Gambar 1).



Gambar 1: [HARDWARE] Hasil Uji Coba Blink pada STM32 Blackpill via ST-Link

1.2 Percobaan 2: ESP32 dengan ESP-IDF (Blink)

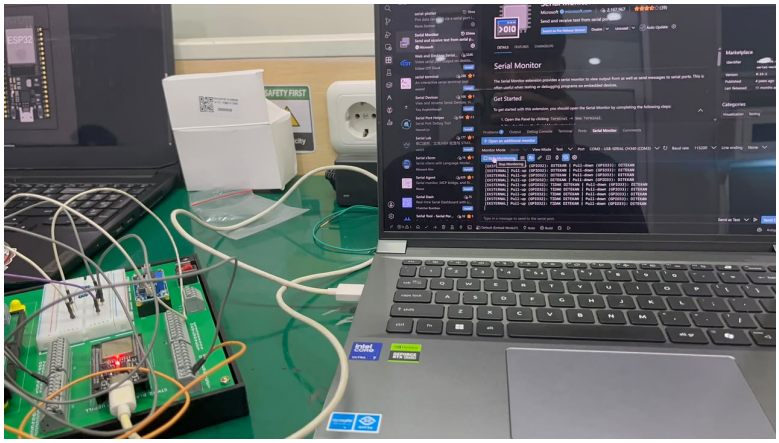
1. Hubungkan ESP32 ke komputer via kabel micro USB.
2. Ubah `platformio.ini` ke board `esp32dev` dan framework `espidf`.
3. Buat `src/main.c` berisi Blink pada GPIO 2 dengan `gpio_set_level()` dan `vTaskDelay()` di `app_main()` (Gambar 2).



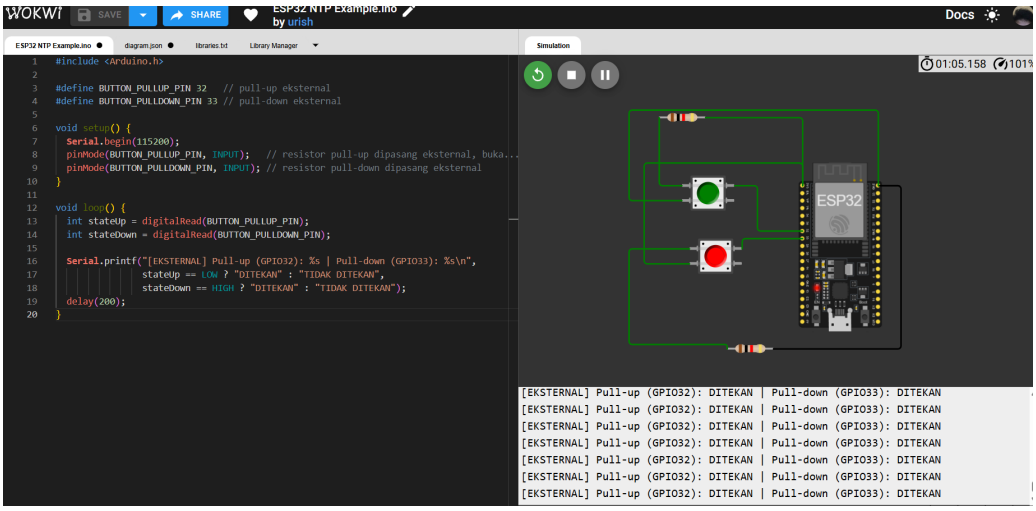
Gambar 2: [HARDWARE] Hasil Uji Coba Blink pada ESP32 Menggunakan Framework ESP-IDF

1.3 Percobaan 3: Pull-up dan Pull-down

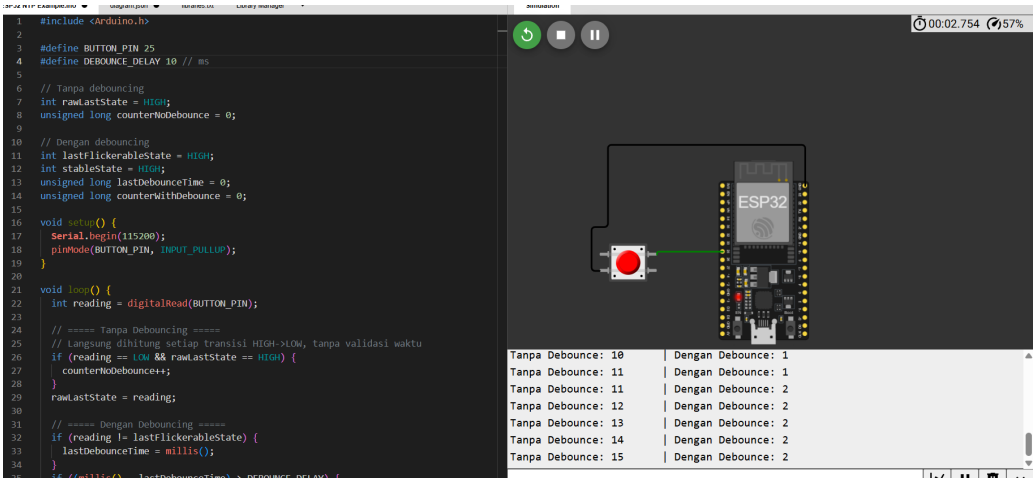
1. Kembalikan framework ke `arduino`.
2. **Eksternal (lab):** pasang pushbutton pull-up (GPIO 32) dan pull-down (GPIO 33) dengan resistor 10 kΩ. Terjadi kesalahan orientasi tombol pull-down (Gambar 3).
3. **Eksternal (simulasi):** ulangi di Wokwi dengan orientasi yang benar (Gambar 4).
4. **Internal (simulasi):** gunakan `INPUT_PULLUP` (GPIO 32) dan `INPUT_PULLDOWN` (GPIO 33) tanpa resistor (Gambar 5).



Gambar 3: [HARDWARE] Hasil Praktikum Percobaan 3 Eksternal di Lab (Salah Orientasi Tombol Pull-down)



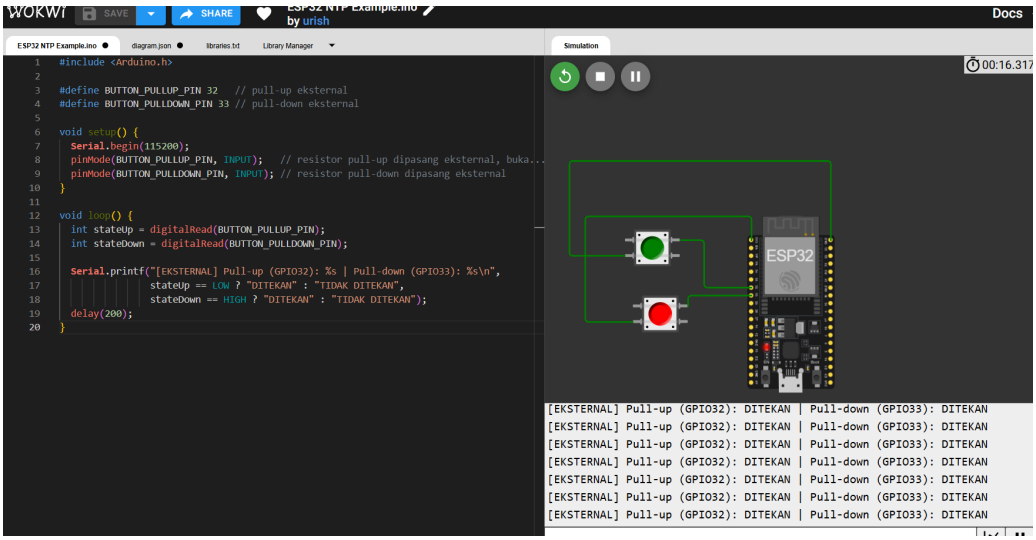
Gambar 4: [SIMULASI WOKWI] Hasil Perbaikan Rangkaian Pull-up dan Pull-down Eksternal



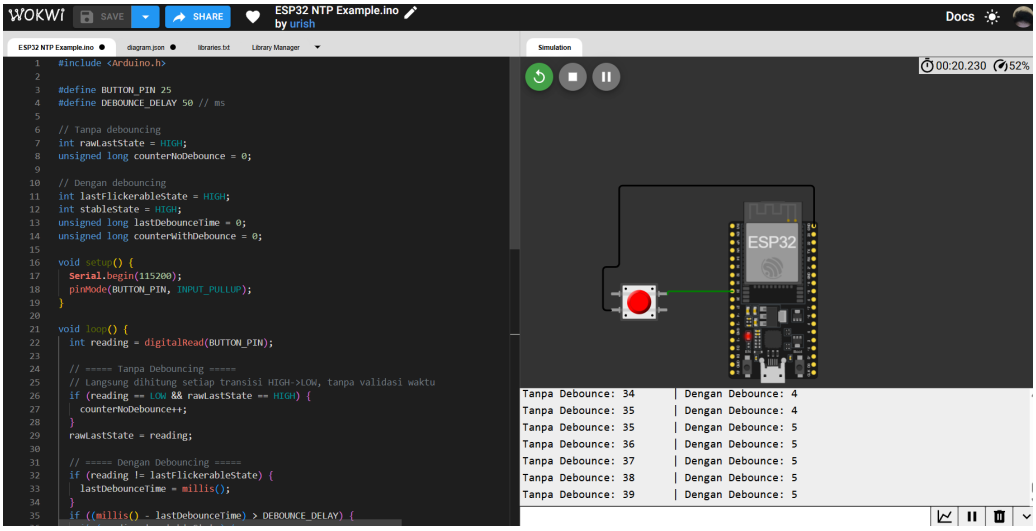
Gambar 5: [SIMULASI WOKWI] Hasil Pengujian Pull-up dan Pull-down Internal ESP32

1.4 Percobaan 4: Debouncing Tombol

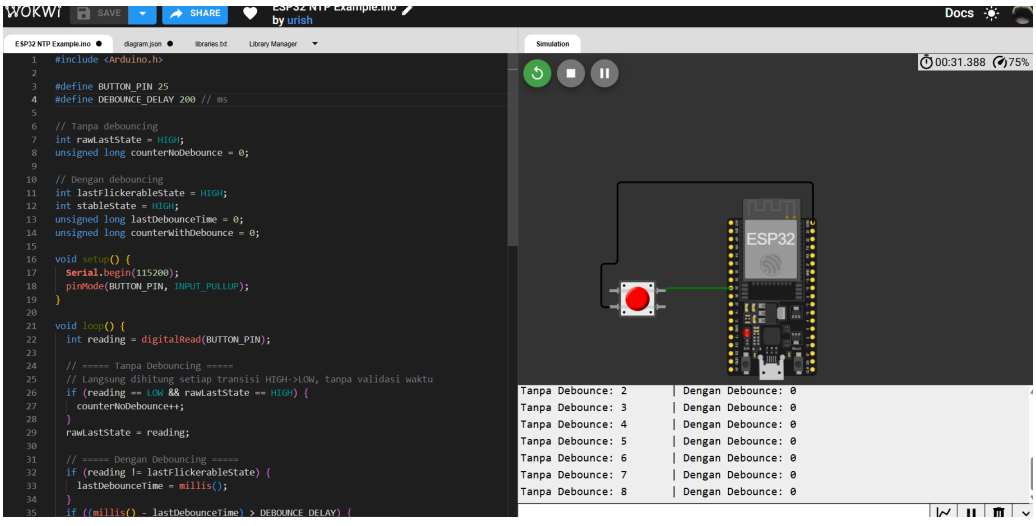
1. Pasang satu pushbutton pada GPIO 25 dengan `INPUT_PULLUP` di Wokwi.
2. Bandingkan `counterNoDebounce` dan `counterWithDebounce` (berbasis `millis()`).
3. Uji delay debounce 10 ms, 50 ms, dan 200 ms (Gambar 6–8).



Gambar 6: [SIMULASI WOKWI] Pengujian Debouncing Tombol dengan Delay 10 ms



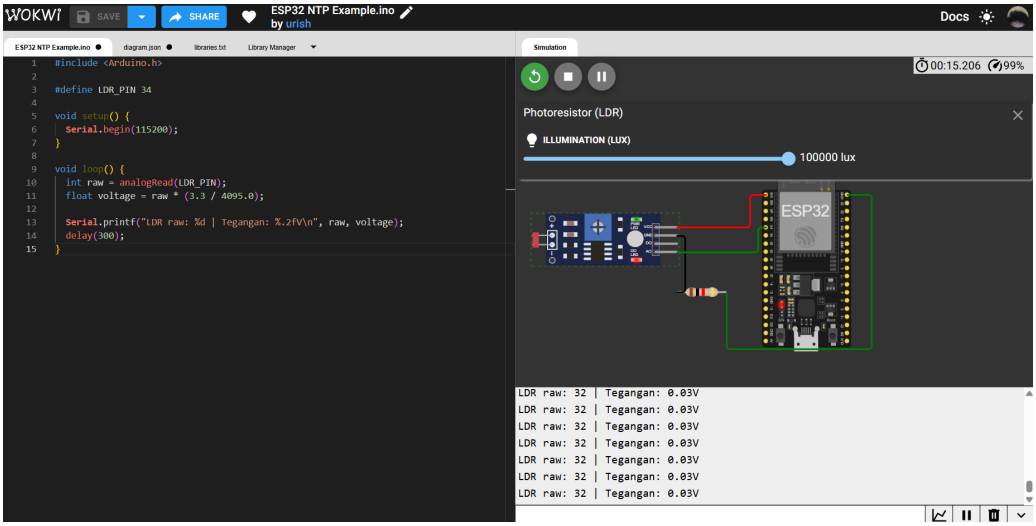
Gambar 7: [SIMULASI WOKWI] Pengujian Debouncing Tombol dengan Delay 50 ms



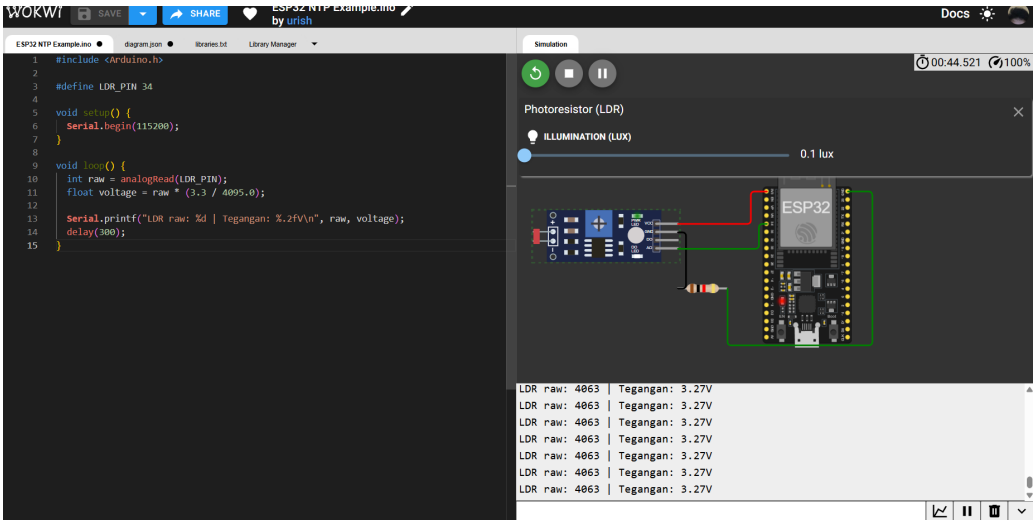
Gambar 8: [SIMULASI WOKWI] Pengujian Debouncing Tombol dengan Delay 200 ms

1.5 Percobaan 5: ADC dengan Sensor LDR

1. Buat pembagi tegangan LDR dan resistor 10 kΩ, titik tengah ke GPIO 34.
2. Baca `analogRead(34)` (12-bit, 0–4095), lalu hitung $V = raw \times (3{,}3/4095)$.
3. Uji pada dua kondisi di Wokwi: terang penuh 100000 lux (Gambar 9) dan gelap penuh 0,1 lux (Gambar 10).



Gambar 9: [SIMULASI WOKWI] Pembacaan ADC Sensor LDR pada Kondisi Terang Penuh (100000 lux)



Gambar 10: [SIMULASI WOKWI] Pembacaan ADC Sensor LDR pada Kondisi Gelap Penuh (0,1 lux)

2 Analisis Hasil Percobaan

2.1 Analisis Percobaan 1: STM32 Blackpill

Program Blink berhasil dijalankan melalui ST-Link V2. LED onboard pada PC13 bersifat *active-low*, sehingga logika LOW menyalakan LED dan HIGH mematikannya. Framework Arduino (STM32duino) memudahkan kontrol pin lewat `digitalWrite()` tanpa konfigurasi register RCC atau clock secara manual.

2.2 Analisis Percobaan 2: ESP32 (ESP-IDF)

Berbeda dengan Arduino yang memakai `setup()` dan `loop()`, ESP-IDF berjalan di atas FreeRTOS dengan fungsi utama `app_main()`. Fungsi `vTaskDelay()` menempatkan task pada kondisi *blocked*, sehingga CPU tidak terjebak dalam *busy-waiting* dan dapat menjalankan task lain.

2.3 Analisis Percobaan 3: Pull-up dan Pull-down

- **Kendala lab:** tombol pull-down (GPIO 33) terbaca HIGH terus-menerus karena tombol 4 kaki terpasang berputar 90°, sehingga kaki yang saling terhubung menyambungkan pin langsung ke 3.3V.
- **Eksternal (perbaikan):** setelah orientasi benar, tombol pull-up bernilai HIGH saat dilepas dan LOW saat ditekan, sedangkan pull-down bernilai LOW saat dilepas dan HIGH saat ditekan.
- **Internal:** `INPUT_PULLUP` dan `INPUT_PULLDOWN` memberikan logika yang sama dengan rangkaian eksternal tanpa resistor tambahan.

2.4 Analisis Percobaan 4: Debouncing

Kontak mekanis tombol menimbulkan *bouncing* saat ditekan atau dilepas.

- `counterNoDebounce` bertambah lebih dari satu kali per penekanan karena membaca osilasi sinyal.
- `counterWithDebounce` menyaring osilasi tersebut sehingga bertambah satu kali per penekanan stabil.
- Delay 10 ms masih berisiko pada tombol yang sudah aus, 50 ms paling seimbang, dan 200 ms membuat respon terasa lambat. Pada Gambar 8, counter dengan debounce tetap bernilai 0 karena penekanan yang lebih singkat dari 200 ms tidak terhitung.

2.5 Analisis Percobaan 5: ADC Sensor LDR

Pada rangkaian ini LDR berada di sisi ground, sehingga tegangan GPIO 34 turun saat cahaya terang dan naik saat gelap.

- **Terang penuh (100000 lux, Gambar 9):** resistansi LDR kecil, *raw* sekitar 32 dengan tegangan sekitar 0,03 V.
- **Gelap penuh (0,1 lux, Gambar 10):** resistansi LDR besar, *raw* sekitar 4063 dengan tegangan sekitar 3,27 V.

ADC membaca tegangan analog dalam 4096 tingkat kuantisasi, sedangkan input digital hanya membaca dua kondisi biner.

3 Hasil Tugas Modul

3.1 Deskripsi dan Rangkaian Simulasi

Tugas modul mengimplementasikan matriks tombol 3×3 untuk mengendalikan 9 LED pada posisi yang bersesuaian, lengkap dengan debouncing pada tiap tombol.

- **Pin baris tombol (output):** Row 1 = GPIO 13, Row 2 = GPIO 12, Row 3 = GPIO 14.
- **Pin kolom tombol (input pull-up):** Col 1 = GPIO 27, Col 2 = GPIO 26, Col 3 = GPIO 25.
- **Pin LED (output):** GPIO 15, 2, 4, 16, 17, 5, 18, 19, dan 21.



Gambar 11: [SIMULASI WOKWI] Hasil Pengujian Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3

3.2 Source Code Program

Listing 1: Source Code Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3 dengan Debouncing

```
#include <Arduino.h>

const int rowPins[3] = {13, 12, 14};
const int colPins[3] = {27, 26, 25};

// Pemetaan 9 LED (matriks 3x3)
const int ledPins[3][3] = {
  {15,  2,  4},
  {16, 17,  5},
  {18, 19, 21}
};

int buttonState[3][3];
int lastButtonState[3][3];
unsigned long lastDebounceTime[3][3];
const unsigned long debounceDelay = 50; // jeda stabil 50 ms

void setup() {
  Serial.begin(115200);
```

```
20
21    for (int r = 0; r < 3; r++) {
22      pinMode(rowPins[r], OUTPUT);
23      digitalWrite(rowPins[r], HIGH);
24    }
25    for (int c = 0; c < 3; c++) {
26      pinMode(colPins[c], INPUT_PULLUP);
27    }
28    for (int r = 0; r < 3; r++) {
29      for (int c = 0; c < 3; c++) {
30        pinMode(ledPins[r][c], OUTPUT);
31        digitalWrite(ledPins[r][c], LOW);
32        buttonState[r][c] = HIGH;
33        lastButtonState[r][c] = HIGH;
34        lastDebounceTime[r][c] = 0;
35      }
36    }
37  }
38
39  void loop() {
40    for (int r = 0; r < 3; r++) {
41      digitalWrite(rowPins[r], LOW);
42
43      for (int c = 0; c < 3; c++) {
44        int reading = digitalRead(colPins[c]);
45
46        if (reading != lastButtonState[r][c]) {
47          lastDebounceTime[r][c] = millis();
48        }
49
50        if ((millis() - lastDebounceTime[r][c]) > debounceDelay) {
51          if (reading != buttonState[r][c]) {
52            buttonState[r][c] = reading;
53
54            if (buttonState[r][c] == LOW) {
55              digitalWrite(ledPins[r][c], HIGH);
56              Serial.printf("Tombol [%d,%d] Ditekan -> LED [%d,%d] NYALA\n",
57                            r + 1, c + 1, r + 1, c + 1);
58            } else {
59              digitalWrite(ledPins[r][c], LOW);
60              Serial.printf("Tombol [%d,%d] Dilepas -> LED [%d,%d] MATI\n",
61                            r + 1, c + 1, r + 1, c + 1);
62            }
63          }
64        }
65        lastButtonState[r][c] = reading;
66      }
67
68      digitalWrite(rowPins[r], HIGH);
69    }
70  }
```

3.3 Jawaban Pertanyaan Analisis Tugas Modul

1. **Mengapa debouncing tetap diperlukan pada matrix pushbutton ini walaupun hanya satu tombol yang ditekan?**
   Karena *bouncing* terjadi di dalam saklar itu sendiri saat pelat logam bersentuhan, bukan akibat banyaknya tombol. Scanning matriks berjalan sangat cepat (orde mikrodetik), sehingga tanpa filter waktu, getaran tersebut terbaca sebagai rentetan penekanan yang membuat LED berkedip dan status tidak stabil.

2. **Bagaimana konfigurasi pull-up/pull-down pada kolom, kondisi saat tombol tidak ditekan, dan bahaya *floating pin*?**
   Kolom memakai pull-up internal (`INPUT_PULLUP`), sehingga saat tombol tidak ditekan pin bernilai HIGH, dan saat ditekan baris aktif (LOW) menarik kolom ke LOW. Pin yang dibiarkan mengambang (*floating*) memiliki impedansi input CMOS yang sangat tinggi sehingga mudah menangkap derau listrik di sekitarnya. Akibatnya logika pin berubah-ubah dan memicu pembacaan penekanan palsu (*ghost trigger*).

3. **Apa perbedaan membaca GPIO sebagai digital input dan analog input (ADC)?**
   Digital input hanya membaca dua kondisi diskret (0 atau 1) berdasarkan ambang tegangan logika (V_{IL} dan V_{IH}). ADC 12-bit pada ESP32 mengubah tegangan 0–3,3 V menjadi